CS212 Solutions: GUI, Events, and Exceptions (Lectures 16–18)

CS212

Multiple Choice Questions

1. What best describes a GUI program?

   A. It executes statements strictly from top to bottom
   B. It repeatedly loops through all code continuously
   C. It waits for events and responds when they occur
   D. It only runs when the user presses a button

   Answer: C

   Explanation: GUI programs are event-driven.

2. What is a callback?

   A. A method that is called immediately when defined
   B. A function passed to be executed later when an event occurs
   C. A function that returns another function
   D. A method that runs automatically at program start

   Answer: B

   Explanation: A callback is executed later in response to an event.

3. Why do we use a `JPanel` inside a `BorderLayout`?

   A. To improve performance
   B. Because BorderLayout cannot display buttons
   C. To allow multiple components in one region
   D. To automatically resize all components

   Answer: C

4. What does `text.trim()` do?

   A. Removes all spaces
   B. Removes leading and trailing whitespace
   C. Splits the string
   D. Converts to lowercase

Answer: B

5. What does `text.split("\\s+")` do?

   A. Splits into characters
   B. Removes whitespace
   C. Splits at commas
   D. Splits into words using whitespace

   Answer: D

6. When an exception is thrown, what happens?

   A. Program always crashes
   B. Method continues normally
   C. Execution pauses
   D. Execution stops and jumps to a catch block

   Answer: D

7. Where should computations be performed?

   A. GUI class
   B. Model class
   C. Event listener
   D. Main method

   Answer: B

8. What is the role of the GUI?

   A. Perform computations
   B. Store data
   C. Handle interaction and display
   D. Manage files

   Answer: C

9. Why separate file IO into its own class?

   A. Avoid exceptions
   B. Reduce memory
   C. Simplify layout
   D. Separate responsibilities

   Answer: D

10. What does a `throws` clause do?

    A. Handles exception
    B. Prevents exception
    C. Declares it may pass to caller
    D. Logs error

    Answer: C

11. What does `frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` do?

    A. Closes all programs
    B. Hides the window
    C. Defines behavior when window closes
    D. Restarts program

    Answer: C

12. What is the purpose of `addActionListener`?

    A. Draw components
    B. Attach behavior to event
    C. Create button
    D. Resize components

    Answer: B

13. What does `new JScrollPane(area)` do?

    A. Creates window
    B. Wraps component for scrolling
    C. Changes font
    D. Adds buttons

    Answer: B

14. What is the default layout of a `JPanel`?

    A. BorderLayout
    B. GridLayout
    C. FlowLayout
    D. Null

    Answer: C

15. What does `setPreferredSize()` do?

A. Forces size

B. Suggests size

C. Resizes frame

D. Removes layout

Answer: B

Programming Problem 1

Write a GUI with:

- A button labeled 'Click Me"
- A label initially showing 'Ready"

When the button is clicked, the label should change to "Clicked".

Solution: Complete Class

```
public class SimpleClickGUI {

    private JLabel label;

    public SimpleClickGUI() {
        JFrame frame = new JFrame("Simple GUI");

        JButton button = new JButton("Click Me");
        label = new JLabel("Ready");

        button.addActionListener(e -> handleClick());

        frame.setLayout(new FlowLayout());
        frame.add(button);
        frame.add(label);

        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }

    private void handleClick() {
        label.setText("Clicked");
    }

    public static void main(String[] args) {
        new SimpleClickGUI();
    }
}
```

Explanation: The GUI creates components and attaches a callback that updates the label.

Programming Problem 2

Write a GUI with:

- A JTextField
- A button “Show Length”
- A label

Solution: Complete Class

```
public class LengthGUI {

    private JTextField field;
    private JLabel label;

    public LengthGUI() {
        JFrame frame = new JFrame("Length GUI");

        field = new JTextField(10);
        JButton button = new JButton("Show Length");
        label = new JLabel("");

        button.addActionListener(e -> handleLength());

        frame.setLayout(new FlowLayout());
        frame.add(field);
        frame.add(button);
        frame.add(label);

        frame.setSize(350, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }

    private void handleLength() {
        String text = field.getText();
        label.setText("" + text.length());
    }

    public static void main(String[] args) {
        new LengthGUI();
    }
}
```

Explanation: The GUI retrieves user input and updates the label based on the computed result.

Programming Problem 3

Design an application with:

- A JTextArea

- A button “Longest Word”
- A label

Solution: Complete Classes
Model Class

```
public class TextModel {

    public String longestWord(String text) throws Exception {
        text = text.trim();

        if (text.isEmpty()) {
            throw new Exception("No words");
        }

        String[] words = text.split("\\s+");
        String longest = words[0];

        for (String w : words) {
            if (w.length() > longest.length()) {
                longest = w;
            }
        }

        return longest;
    }

}
```

GUI Class

```
public class LongestWordGUI {

    private JTextArea area;
    private JLabel label;
    private TextModel model;

    public LongestWordGUI() {
        model = new TextModel();

        JFrame frame = new JFrame("Longest Word");

        area = new JTextArea(5, 20);
        JButton button = new JButton("Longest Word");
        label = new JLabel("");

        button.addActionListener(e -> handleLongest());

        frame.setLayout(new BorderLayout());

        frame.add(new JScrollPane(area), BorderLayout.CENTER);

        JPanel top = new JPanel();
        top.add(button);
```

```
        frame.add(top, BorderLayout.NORTH);
        frame.add(label, BorderLayout.SOUTH);

        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }

    private void handleLongest() {
        try {
            String result = model.longestWord(area.getText());
            label.setText(result);
        } catch (Exception e) {
            label.setText("Error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        new LongestWordGUI();
    }

}
```

Explanation: The model performs computation and throws exceptions. The GUI handles user interaction and displays results.